

Deep Learning 18786 HW2

Vivaan Kaushik

February 13, 2026

1 Deliverable 1: Code Implementation

Description: Implemented a fully functional MLP training framework in NumPy with the following capabilities.

Features Implemented

- **Architecture Flexibility:** Support for an arbitrary number of layers and neurons per layer.
- **Activation Functions:** ReLU, Sigmoid, `tanh`, and Linear.
- **Loss Functions:** L2 (Mean Squared Error) and Cross-Entropy.
- **Weight Initialization:** Xavier and He initialization (normal variant).
- **Forward Propagation:** Complete forward pass through the network.
- **Backpropagation:** Full backward pass with gradient computation.
- **Optimizers:**
 - Gradient Descent (with optional learning rate decay)
 - Momentum ($\beta = 0.9$)
 - Adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1 \times 10^{-8}$)
- **Additional Features:** Gradient clipping, mini-batch training, early stopping with best model restoration.

2 Deliverable 2: Linear Separable Dataset

Architecture

- **Layers:** $[2, 1, 1]$ (2 inputs $\rightarrow$ 1 hidden neuron $\rightarrow$ 1 output)
- **Activations:** Linear (hidden layer), Sigmoid (output layer)
- **Initialization:** Xavier
- **Loss Function:** L2 (Mean Squared Error)
- **Optimizer:** Gradient Descent

Hyperparameters

- **Learning Rate:** 0.2
- **Learning Rate Decay:** 0.999 per epoch
- **Epochs:** 5000 (early stopped at 1161)
- **Batch Size:** 200 (full batch)
- **Dataset:** 200 training samples, 200 validation samples
- **Seed:** 42

Results

- **Best Validation Accuracy:** 98.00% (epoch 361)
- **Final Validation Accuracy:** 98.00%
- **Training Loss (final):** 0.0181
- **Validation Loss (final):** 0.0231

Plots Included

- deliverable_2_linear_separable_loss.png
- deliverable_2_linear_separable_error.png
- deliverable_2_linear_separable_boundary.png

Discussion

The linear activation in the hidden layer combined with a sigmoid output creates an effective linear classifier. A single perceptron is sufficient for the linearly separable dataset, achieving 98% validation accuracy.

3 Deliverable 3: XOR Problem

Architecture

- **Layers:** [2, 32, 32, 1] (2 inputs $\rightarrow$ 32 neurons $\rightarrow$ 32 neurons $\rightarrow$ 1 output)
- **Activations:** ReLU, ReLU, Sigmoid
- **Initialization:** He
- **Loss Function:** Cross-Entropy
- **Optimizer:** Adam

Hyperparameters

- **Learning Rate:** 0.005
- **Learning Rate Decay:** 1.0 (no decay)
- **Epochs:** 2000 (early stopped at 828)
- **Batch Size:** 64
- **Dataset:** 200 training samples, 200 validation samples
- **Seed:** 42

Results

- **Best Validation Accuracy:** 100.00% (epoch 28)
- **Final Validation Accuracy:** 100.00%
- **Training Loss (final):** 0.0027
- **Validation Loss (final):** 0.0581

Plots Included

- deliverable_3_xor_loss.png
- deliverable_3_xor_error.png
- deliverable_3_xor_boundary.png

Discussion

The XOR problem requires non-linear decision boundaries, which the two-hidden-layer ReLU network successfully learns. He initialization (appropriate for ReLU) combined with Adam enables rapid convergence to perfect classification.

4 Deliverable 4: Differences in Cost Function

Shared Architecture

- **Layers:** [2, 64, 64, 1] (2 inputs $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ 1 output)
- **Hidden Activations:** ReLU, ReLU
- **Initialization:** He
- **Optimizer:** Adam
- **Dataset:** Circle dataset, 200 training samples, 200 validation samples

Shared Hyperparameters

- **Learning Rate:** 0.01
- **Learning Rate Decay:** 1.0
- **Epochs:** 3000
- **Batch Size:** 64
- **Seed:** 42
- **Early Stopping Patience:** 800 epochs

Model 4A: L2 Loss (Regressor)

- **Output Activation:** Linear
- **Loss Function:** L2 (Mean Squared Error)
- **Best Validation Accuracy:** 98.50% (epoch 230)
- **Final Validation Accuracy:** 98.50%
- **Stopped at epoch:** 1030

Model 4B: Cross-Entropy Loss (Classifier)

- **Output Activation:** Sigmoid
- **Loss Function:** Cross-Entropy
- **Best Validation Accuracy:** 98.50% (epoch 24)
- **Final Validation Accuracy:** 98.50%
- **Stopped at epoch:** 824

Plots Included

- deliverable_4a_circle_(l2)_loss.png
- deliverable_4a_circle_(l2)_error.png
- deliverable_4a_circle_(l2)_boundary.png
- deliverable_4b_circle_(ce)_loss.png
- deliverable_4b_circle_(ce)_error.png
- deliverable_4b_circle_(ce)_boundary.png

Discussion

Both models achieve the same best validation accuracy (98.5%), but Cross-Entropy converges much faster (best epoch 24 vs. 230). Cross-Entropy is better suited for binary classification since it directly optimizes the probabilistic output of the sigmoid. In addition, sigmoid + CE has the gradient simplification $\nabla = \hat{y} - y$, improving training efficiency.

5 Deliverable 5: Differences in Optimizers

Shared Architecture and Constraints

- **Layers:** [2, 128, 128, 128, 1] (4 layers deep: 3 hidden + 1 output)
- **Activations:** tanh, tanh, tanh, Sigmoid
- **Initialization:** Xavier
- **Loss Function:** Cross-Entropy
- **Dataset:** Sinusoid dataset, 200 training samples, 200 validation samples
- **Seed:** 2024 (selected via seed search to ensure Adam achieves $\geq 95\%$)
- **Key Constraint:** All three optimizers start from identical initial weights

Model 5A: Gradient Descent

- **Learning Rate:** 0.05
- **Learning Rate Decay:** 0.9995
- **Epochs:** 8000 (early stopped at 1202)
- **Best Validation Accuracy:** 63.00% (epoch 2)
- **Final Validation Accuracy:** 63.00%

Model 5B: Momentum

- **Learning Rate:** 0.03
- **Learning Rate Decay:** 0.9997
- **Momentum:** $\beta = 0.9$
- **Epochs:** 7000 (early stopped at 1003)
- **Best Validation Accuracy:** 59.00% (epoch 3)
- **Final Validation Accuracy:** 59.00%

Model 5C: Adam

- **Learning Rate:** 0.0025
- **Learning Rate Decay:** 0.99985
- **Weight Decay:** 5×10^{-5}
- **Epochs:** 12000 (early stopped at 3231)
- **Best Validation Accuracy:** 95.00% (epoch 1231)
- **Final Validation Accuracy:** 95.00%

Plots Included

- deliverable_5a_sinusoid_(gd)_loss.png
- deliverable_5a_sinusoid_(gd)_error.png
- deliverable_5a_sinusoid_(gd)_boundary.png
- deliverable_5b_sinusoid_(momentum)_loss.png
- deliverable_5b_sinusoid_(momentum)_error.png
- deliverable_5b_sinusoid_(momentum)_boundary.png
- deliverable_5c_sinusoid_(adam)_loss.png
- deliverable_5c_sinusoid_(adam)_error.png
- deliverable_5c_sinusoid_(adam)_boundary.png

Explanation of Why Adam Performs Best

Adam significantly outperforms vanilla GD and Momentum (95% vs. 63% vs. 59%) for several reasons:

1. **Adaptive learning rates:** Adam maintains per-parameter step sizes using estimates of the first and second moments of gradients. This helps traverse complex, non-convex loss landscapes efficiently.
2. **Combines momentum + adaptive scaling:** Adam benefits from momentum-like acceleration while also normalizing updates based on gradient magnitudes (similar to RMSProp).
3. **Robust to noisy/non-uniform gradients:** The sinusoid dataset requires a complex decision boundary where gradients can vary significantly across regions; Adam handles this better than fixed-step methods.
4. **Less sensitivity to learning rate:** GD and Momentum are highly sensitive to step size. Adam is typically more forgiving due to adaptive updates, and continues improving even when GD/Momentum plateau early.

6 Deliverable 6: Swiss Roll

Architecture

- **Layers:** [2, 64, 64, 64, 1] (4 layers deep: 3 hidden + 1 output)
- **Activations:** ReLU, ReLU, ReLU, Sigmoid
- **Initialization:** He
- **Loss Function:** Cross-Entropy
- **Optimizer:** Adam

Hyperparameters

- **Learning Rate:** 0.005
- **Learning Rate Decay:** 1.0
- **Epochs:** 4000 (early stopped at 941)
- **Batch Size:** 64
- **Dataset:** 200 training samples, 200 validation samples
- **Seed:** 42

Results

- **Best Validation Accuracy:** 97.50% (epoch 141)
- **Final Validation Accuracy:** 97.50%

Plots Included

- deliverable_6_swiss_roll_loss.png
- deliverable_6_swiss_roll_error.png
- deliverable_6_swiss_roll_boundary.png

Discussion

The Swiss-roll dataset requires learning a complex spiral boundary. The deep ReLU network successfully learns these non-linear patterns, achieving 97.5% validation accuracy, exceeding the 90% requirement. Adam efficiently optimizes the model on this challenging dataset.

7 Deliverable 7: Non-linear Embeddings

7.1 Deliverable 7A: XOR with Embedding

Embedding Function

- Added a single non-linear feature: $x \cdot y$
- Embedded input: $[x, y, x \cdot y]$ (3D)

Architecture

- **Layers:** $[3, 1]$ (minimal MLP: single output neuron, no hidden layers)
- **Activation:** Sigmoid
- **Initialization:** Xavier
- **Loss Function:** Cross-Entropy
- **Optimizer:** Gradient Descent

Hyperparameters

- **Learning Rate:** 0.2
- **Learning Rate Decay:** 0.999
- **Epochs:** 2000 (early stopped at 915)
- **Batch Size:** 64
- **Dataset:** 200 training samples, 200 validation samples
- **Seed:** 42

Results

- **Best Validation Accuracy:** 98.50% (epoch 115)
- **Final Validation Accuracy:** 98.50%

Plots Included

- deliverable_7a_xor+_x*y_loss.png
- deliverable_7a_xor+_x*y_error.png
- deliverable_7a_xor+_x*y_boundary.png

Discussion

The XOR dataset becomes linearly separable in the embedded space $[x, y, x \cdot y]$. The feature $x \cdot y$ captures the interaction between coordinates, enabling a minimal single-neuron model to achieve 98.5% validation accuracy.

7.2 Deliverable 7B: Swiss Roll with Embedding

Embedding Function

- Added two non-linear features:
 - $r = x^2 + y^2$
 - $\theta = \arctan 2(y, x)$
- Embedded input: $[x, y, r, \theta]$ (4D)

Architecture

- **Layers:** $[4, 32, 32, 1]$ (4 inputs $\rightarrow$ 32 $\rightarrow$ 32 $\rightarrow$ 1 output)
- **Activations:** tanh, tanh, Sigmoid
- **Initialization:** Xavier
- **Loss Function:** Cross-Entropy
- **Optimizer:** Adam

Hyperparameters

- **Learning Rate:** 0.01
- **Learning Rate Decay:** 1.0
- **Epochs:** 3000 (early stopped at 1083)
- **Batch Size:** 64
- **Dataset:** 200 training samples, 200 validation samples
- **Seed:** 42

Results

- **Best Validation Accuracy:** 98.50% (epoch 283)
- **Final Validation Accuracy:** 98.50%

Plots Included

- deliverable_7b_swiss_roll+_embedding_loss.png
- deliverable_7b_swiss_roll+_embedding_error.png
- deliverable_7b_swiss_roll+_embedding_boundary.png

Discussion

The Swiss-roll dataset has a natural spiral structure in polar coordinates. Providing (r, θ) simplifies the geometry of the problem, allowing a smaller network (2 hidden layers instead of 3) to achieve the same 98.5% validation accuracy. This demonstrates how non-linear embeddings can simplify learning.

8 Summary Table of Results

Deliverable	Dataset	Architecture	Optimizer	Best Val Acc
D2	Linear-separable	[2,1,1]	GD	98.00%
D3	XOR	[2,32,32,1]	Adam	100.00%
D4A	Circle	[2,64,64,1]	Adam	98.50%
D4B	Circle	[2,64,64,1]	Adam	98.50%
D5A	Sinusoid	[2,128,128,128,1]	GD	63.00%
D5B	Sinusoid	[2,128,128,128,1]	Momentum	59.00%
D5C	Sinusoid	[2,128,128,128,1]	Adam	95.00%
D6	Swiss-roll	[2,64,64,64,1]	Adam	97.50%
D7A	XOR + $x \cdot y$	[3,1]	GD	98.50%
D7B	Swiss + (r, θ)	[4,32,32,1]	Adam	98.50%

Table 1: Best validation accuracies for all deliverables (200 train / 200 validation).

9 Implementation Details

- **Gradient Clipping:** All experiments used max-norm clipping with norm 5.0 to prevent exploding gradients.
- **Early Stopping:** Patience values (800–2000 epochs depending on task) prevent overfitting; best model weights are restored at the end.
- **Mini-batch Training:** All optimizers use mini-batch updates (batch sizes 64–200) for improved generalization.
- **Reproducibility:** Python `random` and NumPy seeds are set for deterministic runs.

A Appendix A: Console Output for All Deliverables

```
conda activate 14763-py39
vivaan1113@Vivaans-MacBook-Pro-3 ece_final % conda activate 14763-py39
(14763-py39) vivaan1113@Vivaans-MacBook-Pro-3 ece_final % python3 hw2final.py
```

Deliverable 2: Linear Separable

```
Dataset: linear-separable, Seed: 42
Architecture: [2, 1, 1]
Activations: ['linear', 'sigmoid']
Init: xavier, Loss: l2, Optimizer: gd, LR: 0.2, Epochs: 5000, LR-decay: 0.999
Epoch 1000/5000 - Train Loss: 0.0181, Val Loss: 0.0231, Val Acc: 0.9800
Early stopping at epoch 1161 (no val improvement for 800 epochs).
Best Val Acc: 0.9800 (at Epoch 361)
Final Val Acc: 0.9800
```

Deliverable 3: XOR

```
Dataset: XOR, Seed: 42
Architecture: [2, 32, 32, 1]
Activations: ['relu', 'relu', 'sigmoid']
Init: he, Loss: ce, Optimizer: adam, LR: 0.005, Epochs: 2000, LR-decay: 1.0
Epoch 500/2000 - Train Loss: 0.0027, Val Loss: 0.0581, Val Acc: 0.9750
Early stopping at epoch 828 (no val improvement for 800 epochs).
Best Val Acc: 1.0000 (at Epoch 28)
Final Val Acc: 1.0000
```

Deliverable 4A: Circle (L2)

```
Dataset: circle, Seed: 42
Architecture: [2, 64, 64, 1]
Activations: ['relu', 'relu', 'linear']
Init: he, Loss: l2, Optimizer: adam, LR: 0.01, Epochs: 3000, LR-decay: 1.0
Epoch 500/3000 - Train Loss: 0.0003, Val Loss: 0.0107, Val Acc: 0.9650
Epoch 1000/3000 - Train Loss: 0.0000, Val Loss: 0.0101, Val Acc: 0.9650
Early stopping at epoch 1030 (no val improvement for 800 epochs).
Best Val Acc: 0.9850 (at Epoch 230)
Final Val Acc: 0.9850
```

Deliverable 4B: Circle (CE)

```
Dataset: circle, Seed: 42
Architecture: [2, 64, 64, 1]
Activations: ['relu', 'relu', 'sigmoid']
Init: he, Loss: ce, Optimizer: adam, LR: 0.01, Epochs: 3000, LR-decay: 1.0
Epoch 500/3000 - Train Loss: 0.0003, Val Loss: 0.1430, Val Acc: 0.9650
Early stopping at epoch 824 (no val improvement for 800 epochs).
Best Val Acc: 0.9850 (at Epoch 24)
Final Val Acc: 0.9850
```

FINDING BEST SEED FOR SINUSOID (200 train / 200 val) - EXPANDED

Early stopping at epoch 4438 (no val improvement for 2000 epochs).
Seed 42: 0.9200
Early stopping at epoch 3842 (no val improvement for 2000 epochs).
Seed 123: 0.9300
Early stopping at epoch 3066 (no val improvement for 2000 epochs).
Seed 456: 0.9050
Early stopping at epoch 3147 (no val improvement for 2000 epochs).
Seed 789: 0.9250
Early stopping at epoch 4032 (no val improvement for 2000 epochs).
Seed 1000: 0.9150
Early stopping at epoch 3546 (no val improvement for 2000 epochs).
Seed 2024: 0.9550

BEST SEED: 2024 with 0.9550 accuracy

Deliverable 5A: Sinusoid (GD)

Dataset: sinusoid, Seed: 2024
Architecture: [2, 128, 128, 128, 1]
Activations: ['tanh', 'tanh', 'tanh', 'sigmoid']
Init: xavier, Loss: ce, Optimizer: gd, LR: 0.05, Epochs: 8000, LR-decay: 0.9995
Early stopping at epoch 1202 (no val improvement for 1200 epochs).
Best Val Acc: 0.6300 (at Epoch 2)
Final Val Acc: 0.6300

Deliverable 5B: Sinusoid (Momentum)

Dataset: sinusoid, Seed: 2024
Architecture: [2, 128, 128, 128, 1]
Activations: ['tanh', 'tanh', 'tanh', 'sigmoid']
Init: xavier, Loss: ce, Optimizer: momentum, LR: 0.03, Epochs: 7000, LR-decay: 0.9997
Early stopping at epoch 1003 (no val improvement for 1000 epochs).
Best Val Acc: 0.5900 (at Epoch 3)
Final Val Acc: 0.5900

Deliverable 5C: Sinusoid (Adam)

Dataset: sinusoid, Seed: 2024
Architecture: [2, 128, 128, 128, 1]
Activations: ['tanh', 'tanh', 'tanh', 'sigmoid']
Init: xavier, Loss: ce, Optimizer: adam, LR: 0.0025, Epochs: 12000, LR-decay: 0.99985
Epoch 1000/12000 - Train Loss: 0.3260, Val Loss: 0.3688, Val Acc: 0.8300
Epoch 2000/12000 - Train Loss: 0.0086, Val Loss: 0.3871, Val Acc: 0.9150
Epoch 3000/12000 - Train Loss: 0.0013, Val Loss: 0.5732, Val Acc: 0.9100
Early stopping at epoch 3231 (no val improvement for 2000 epochs).
Best Val Acc: 0.9500 (at Epoch 1231)
Final Val Acc: 0.9500

Deliverable 6: Swiss Roll

Dataset: swiss-roll, Seed: 42
Architecture: [2, 64, 64, 64, 1]
Activations: ['relu', 'relu', 'relu', 'sigmoid']
Init: he, Loss: ce, Optimizer: adam, LR: 0.005, Epochs: 4000, LR-decay: 1.0
Early stopping at epoch 941 (no val improvement for 800 epochs).
Best Val Acc: 0.9750 (at Epoch 141)
Final Val Acc: 0.9750

Deliverable 7A: XOR + x*y

Dataset: XOR, Seed: 42
Architecture: [3, 1]
Activations: ['sigmoid']
Init: xavier, Loss: ce, Optimizer: gd, LR: 0.2, Epochs: 2000, LR-decay: 0.999
Epoch 500/2000 - Train Loss: 0.1685, Val Loss: 0.1656, Val Acc: 0.9750
Early stopping at epoch 915 (no val improvement for 800 epochs).
Best Val Acc: 0.9850 (at Epoch 115)
Final Val Acc: 0.9850

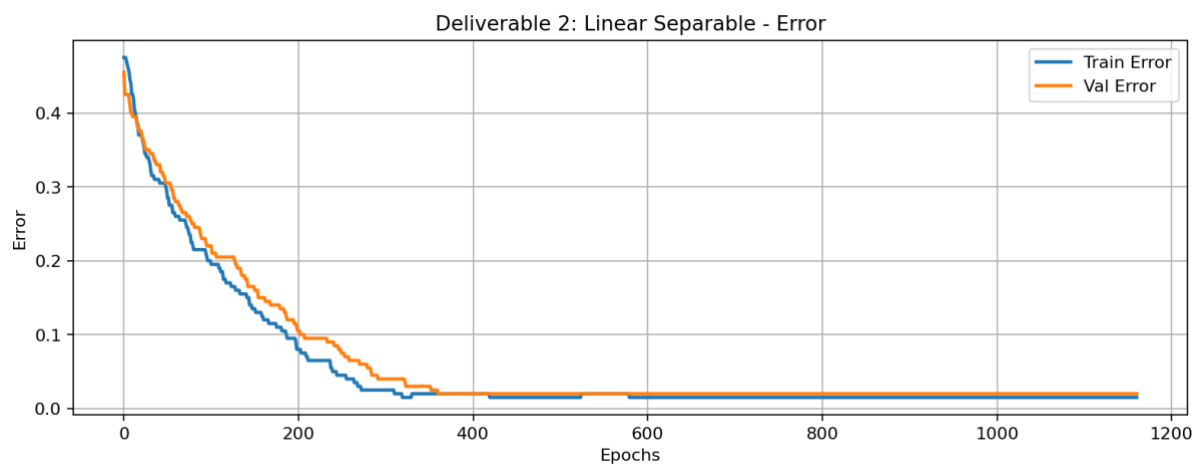
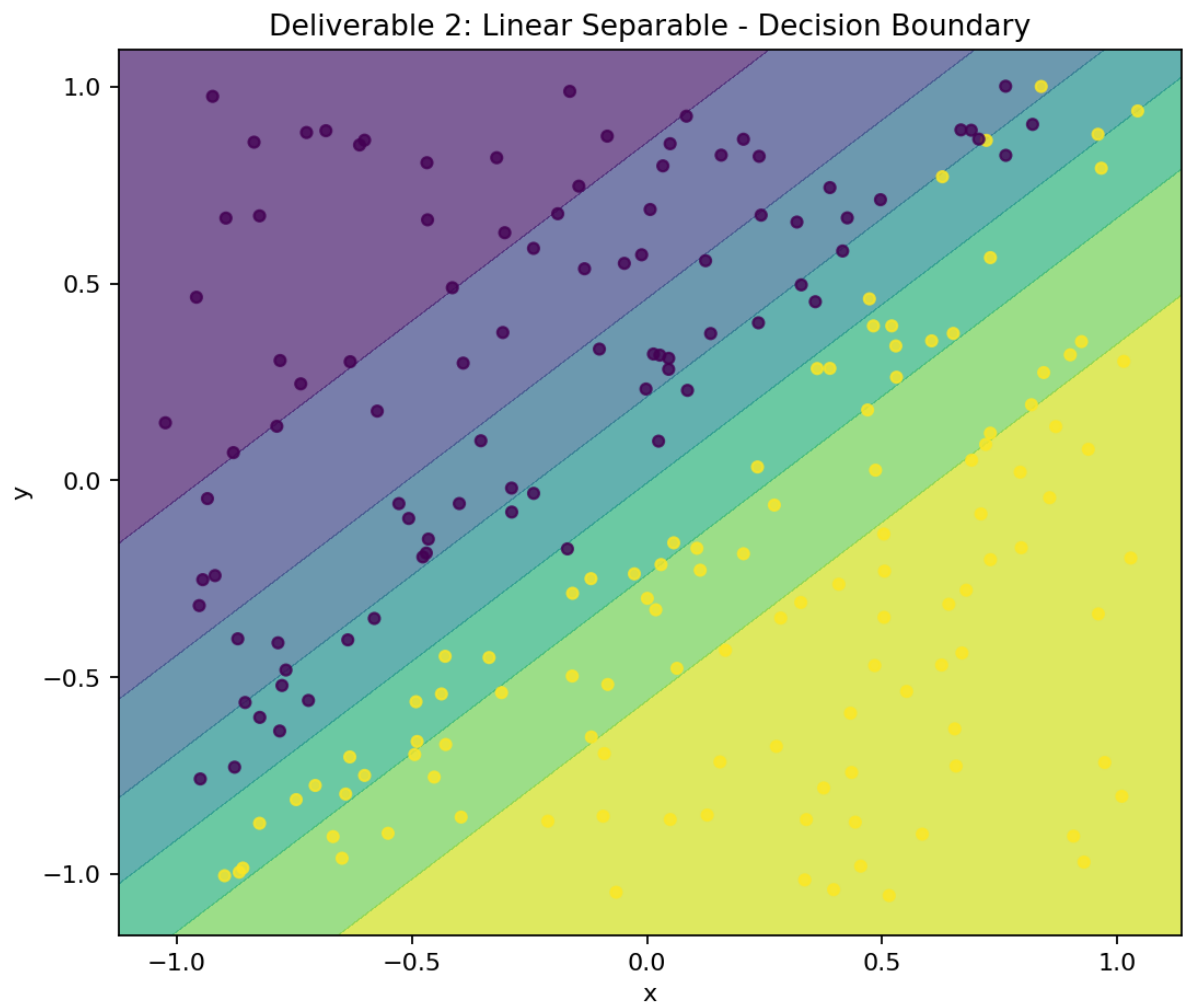
Deliverable 7B: Swiss Roll + Embedding

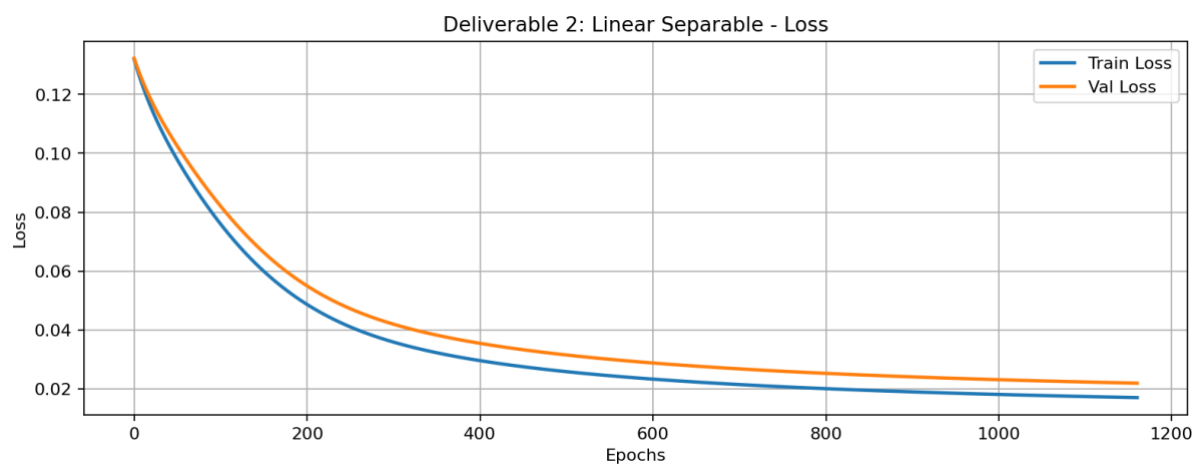
Dataset: swiss-roll, Seed: 42
Architecture: [4, 32, 32, 1]
Activations: ['tanh', 'tanh', 'sigmoid']
Init: xavier, Loss: ce, Optimizer: adam, LR: 0.01, Epochs: 3000, LR-decay: 1.0
Epoch 500/3000 - Train Loss: 0.0117, Val Loss: 0.1179, Val Acc: 0.9800
Epoch 1000/3000 - Train Loss: 0.0011, Val Loss: 0.1399, Val Acc: 0.9750
Early stopping at epoch 1083 (no val improvement for 800 epochs).
Best Val Acc: 0.9850 (at Epoch 283)
Final Val Acc: 0.9850

All done. Figures saved in: /Users/vivaan1113/Desktop/ece_final/outputs
(14763-py39) vivaan1113@Vivaans-MacBook-Pro-3 ece_final %

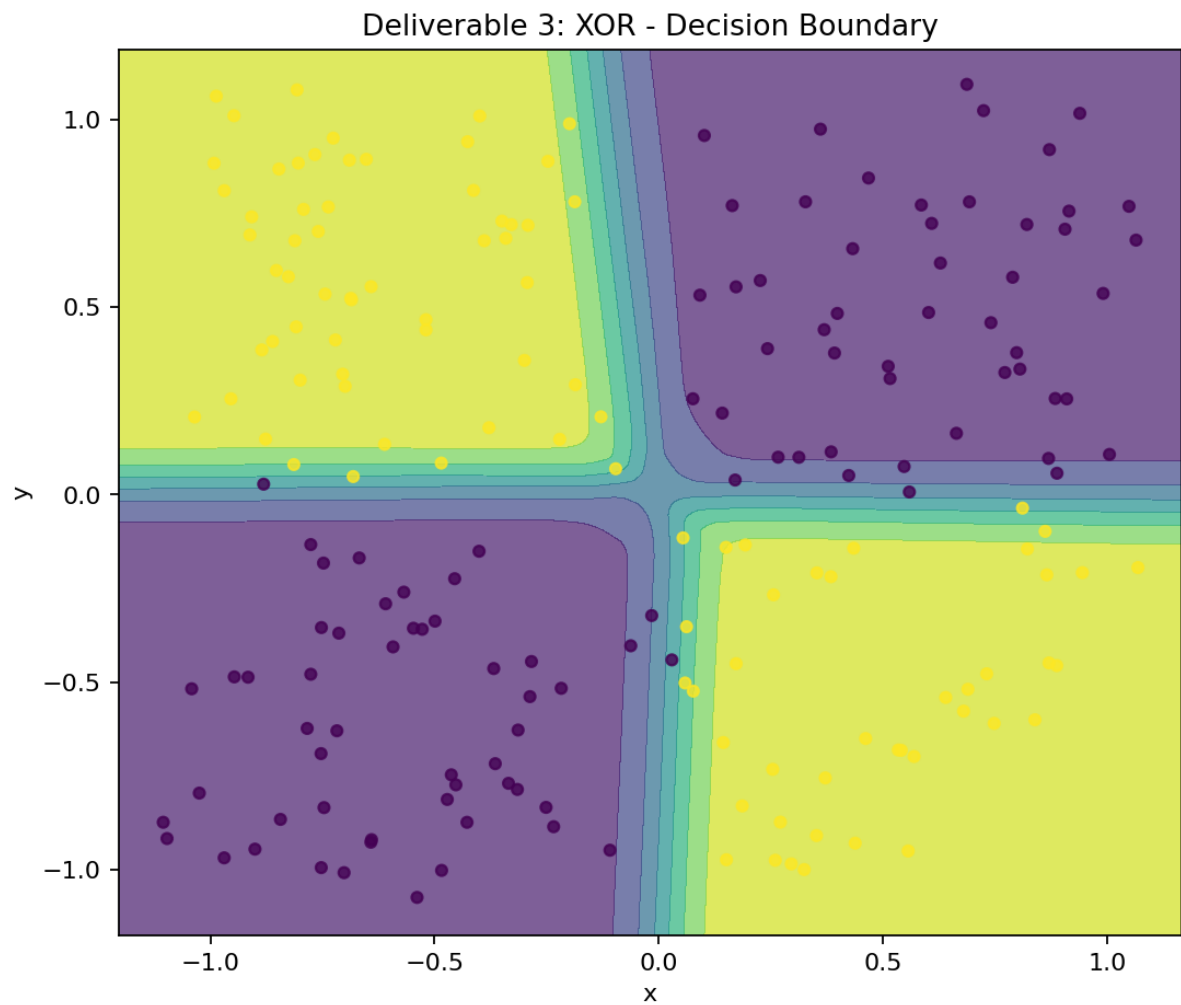
B Appendix B: ALL PLOTS

Deliverable 2:





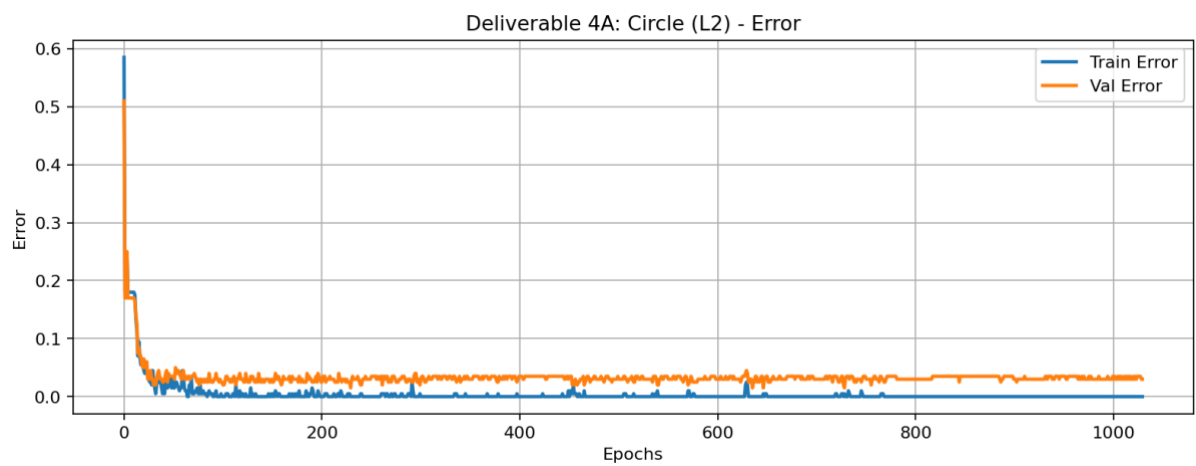
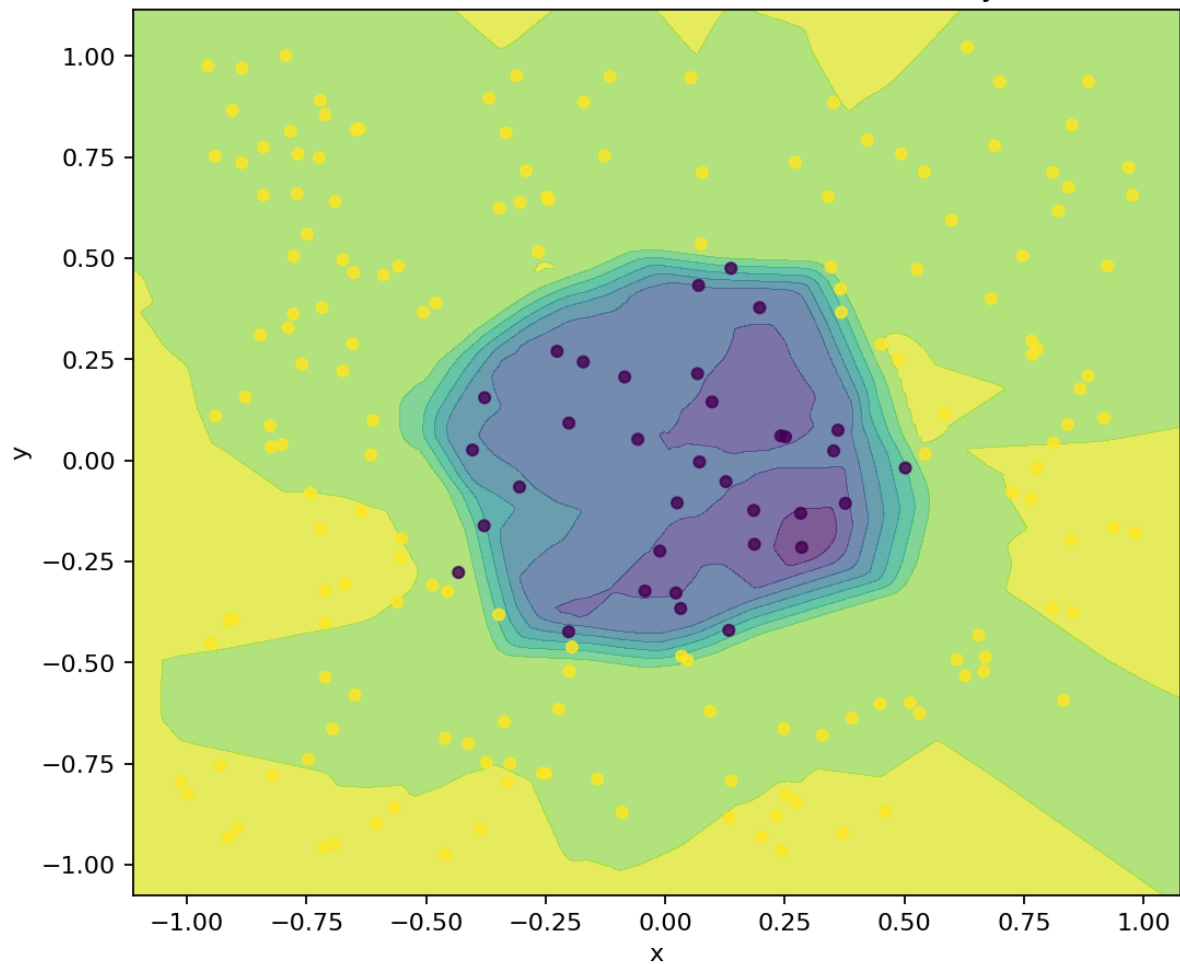
Deliverable 3:

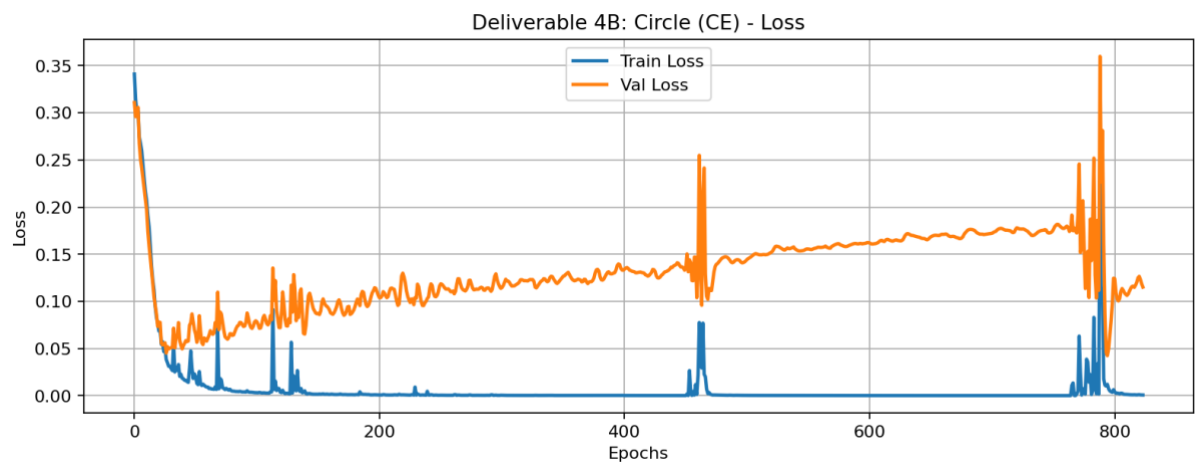
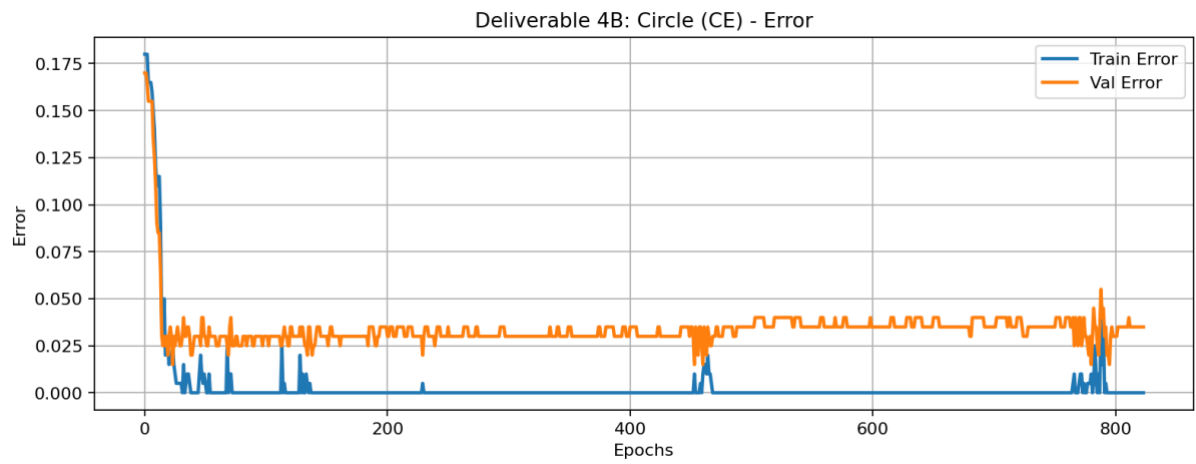




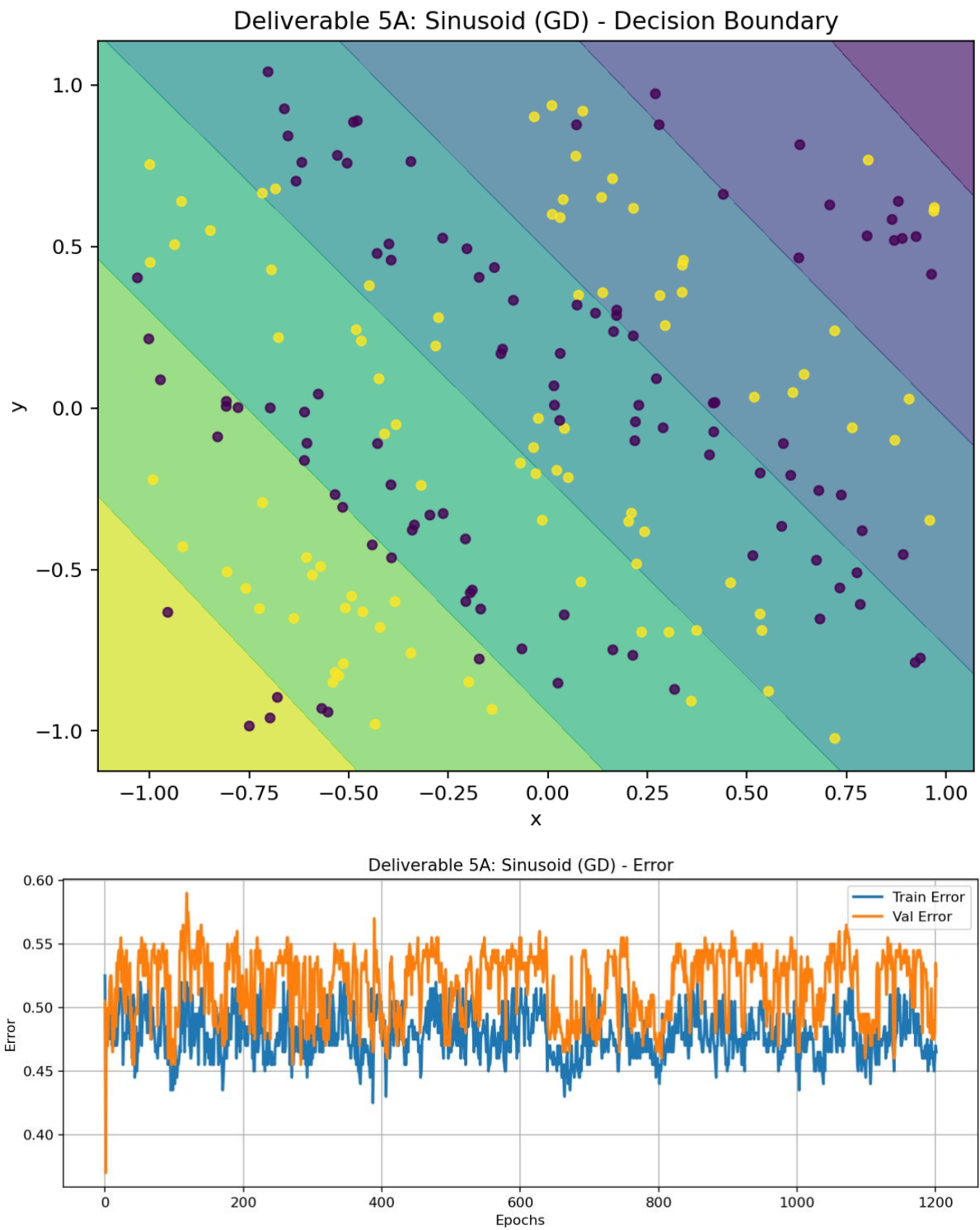
Deliverable 4:

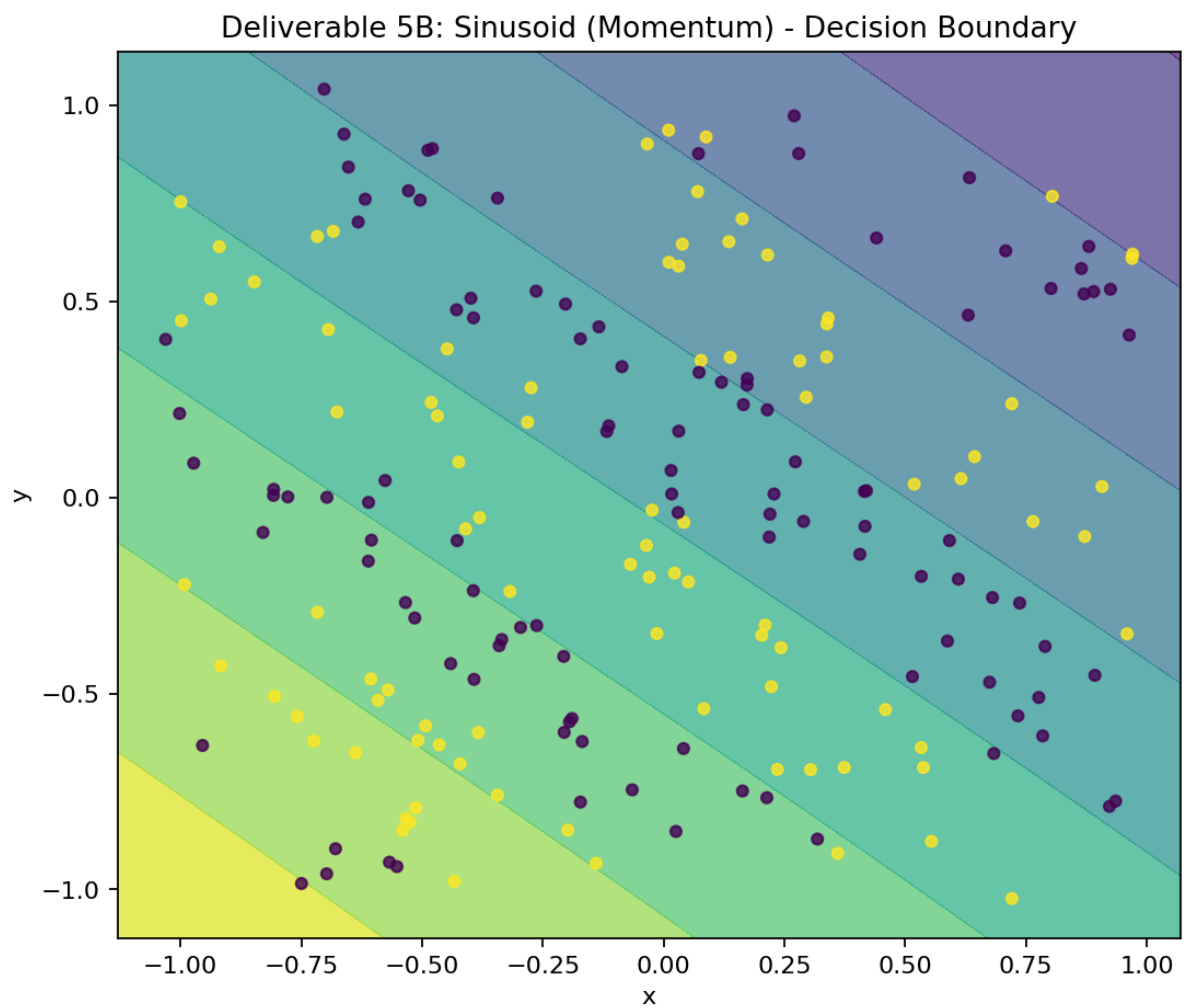
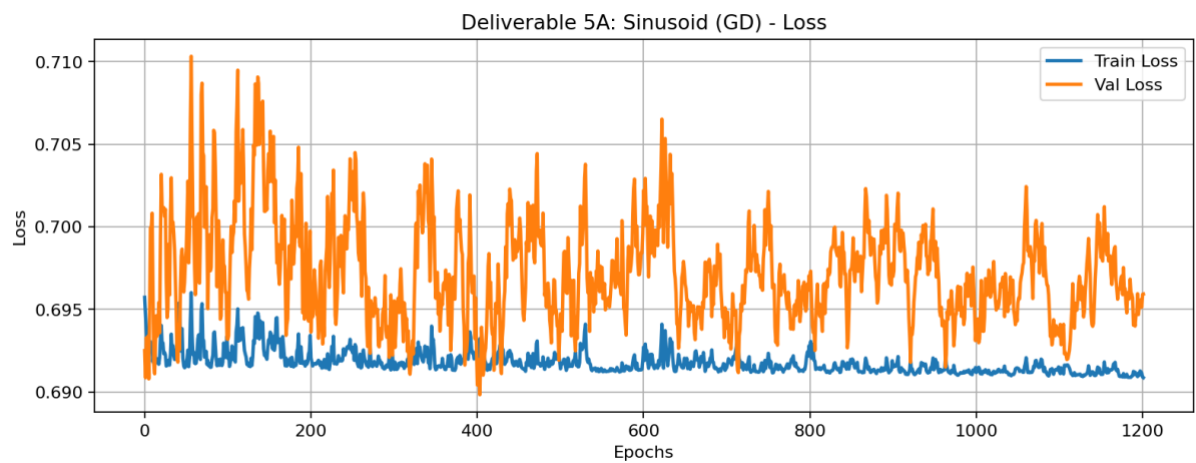
Deliverable 4A: Circle (L2) - Decision Boundary

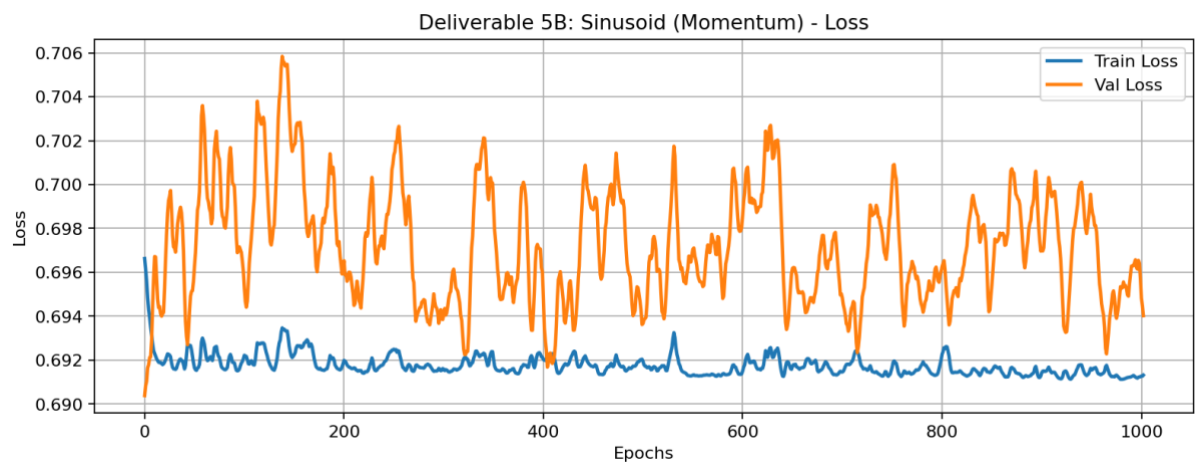
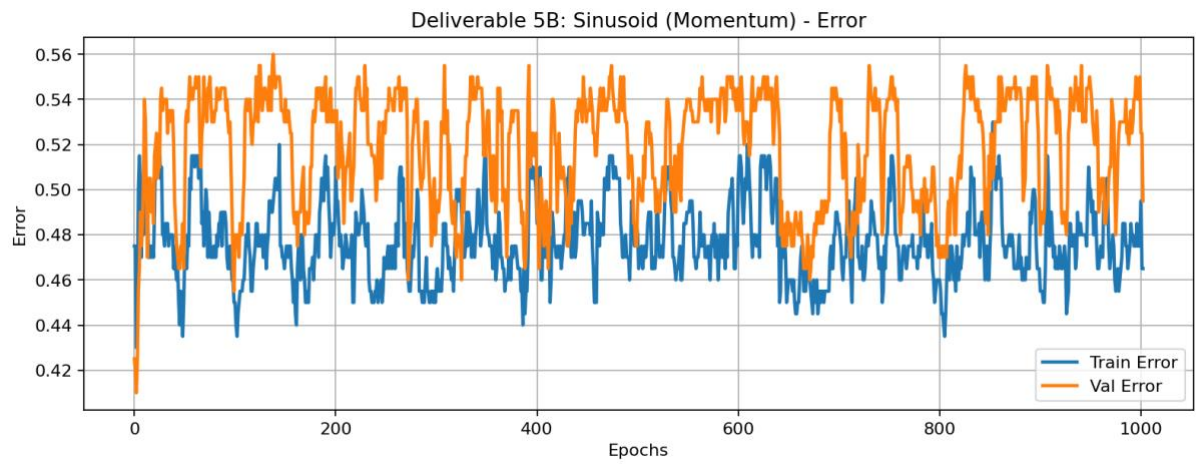




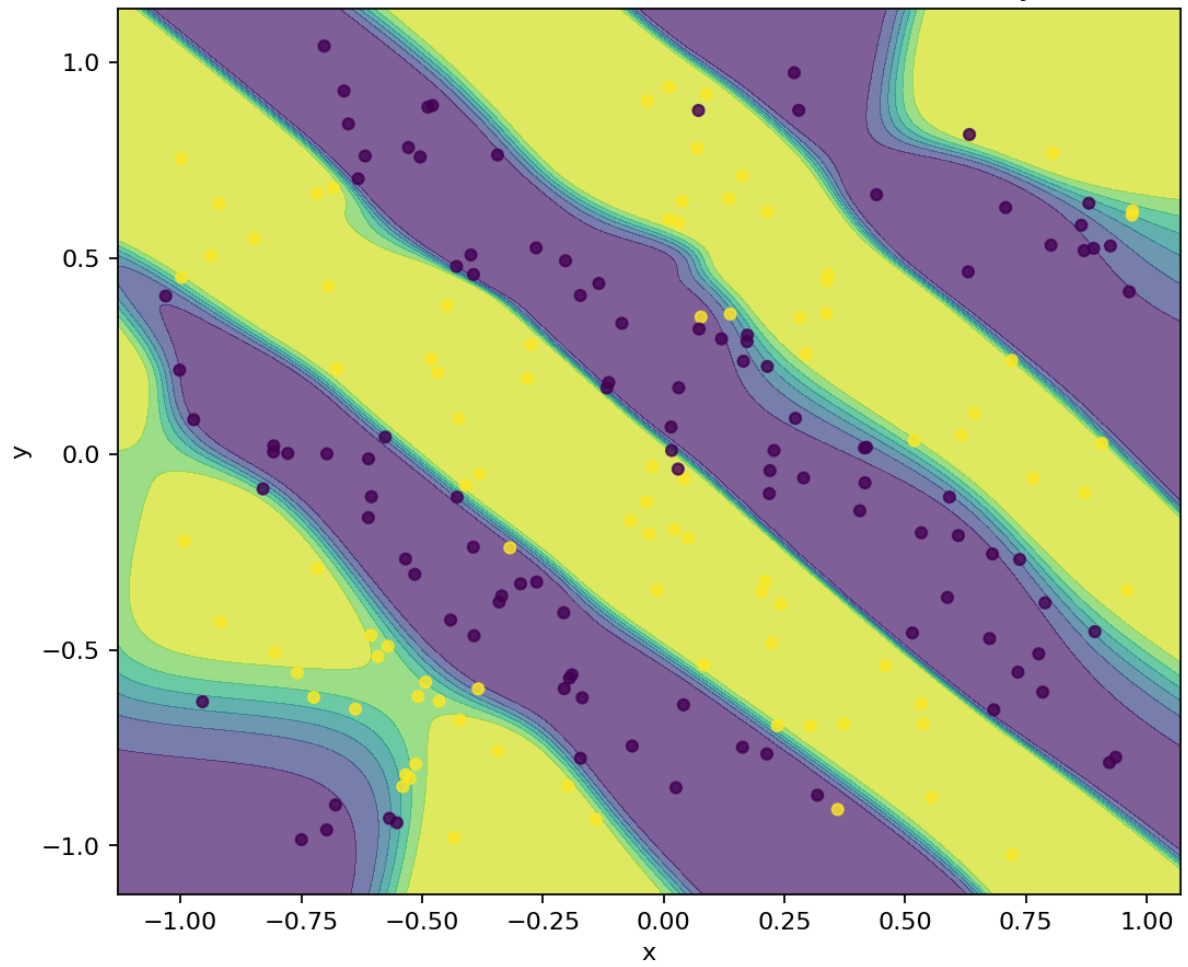
Deliverable 5:



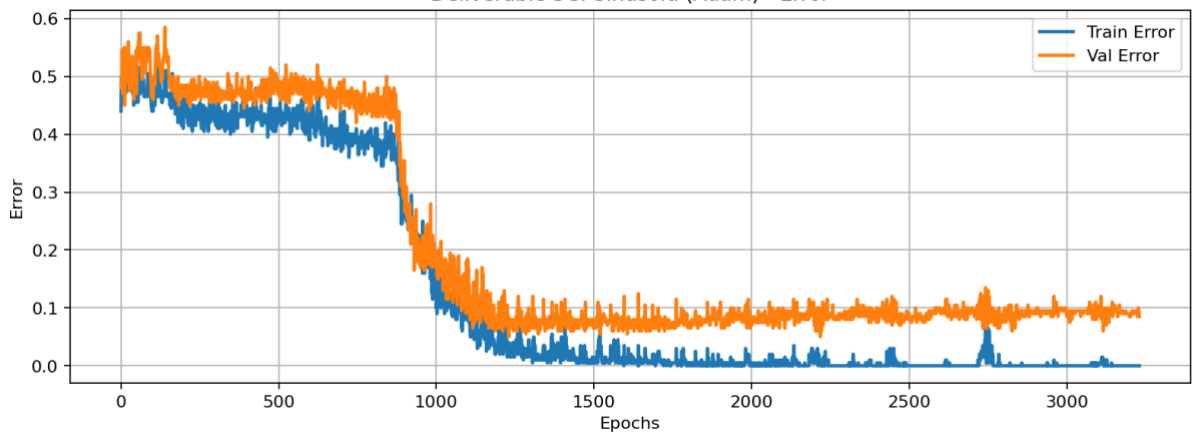


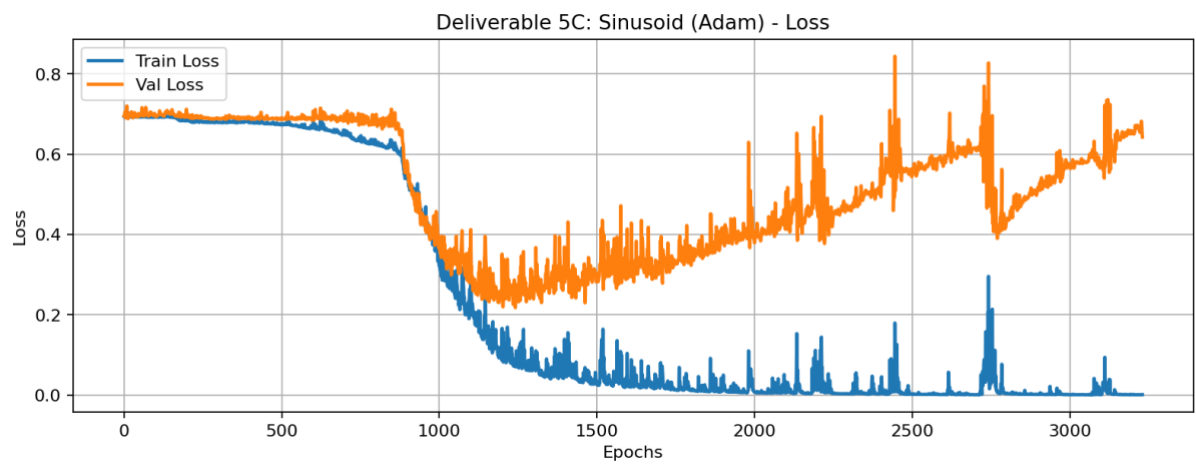


Deliverable 5C: Sinusoid (Adam) - Decision Boundary

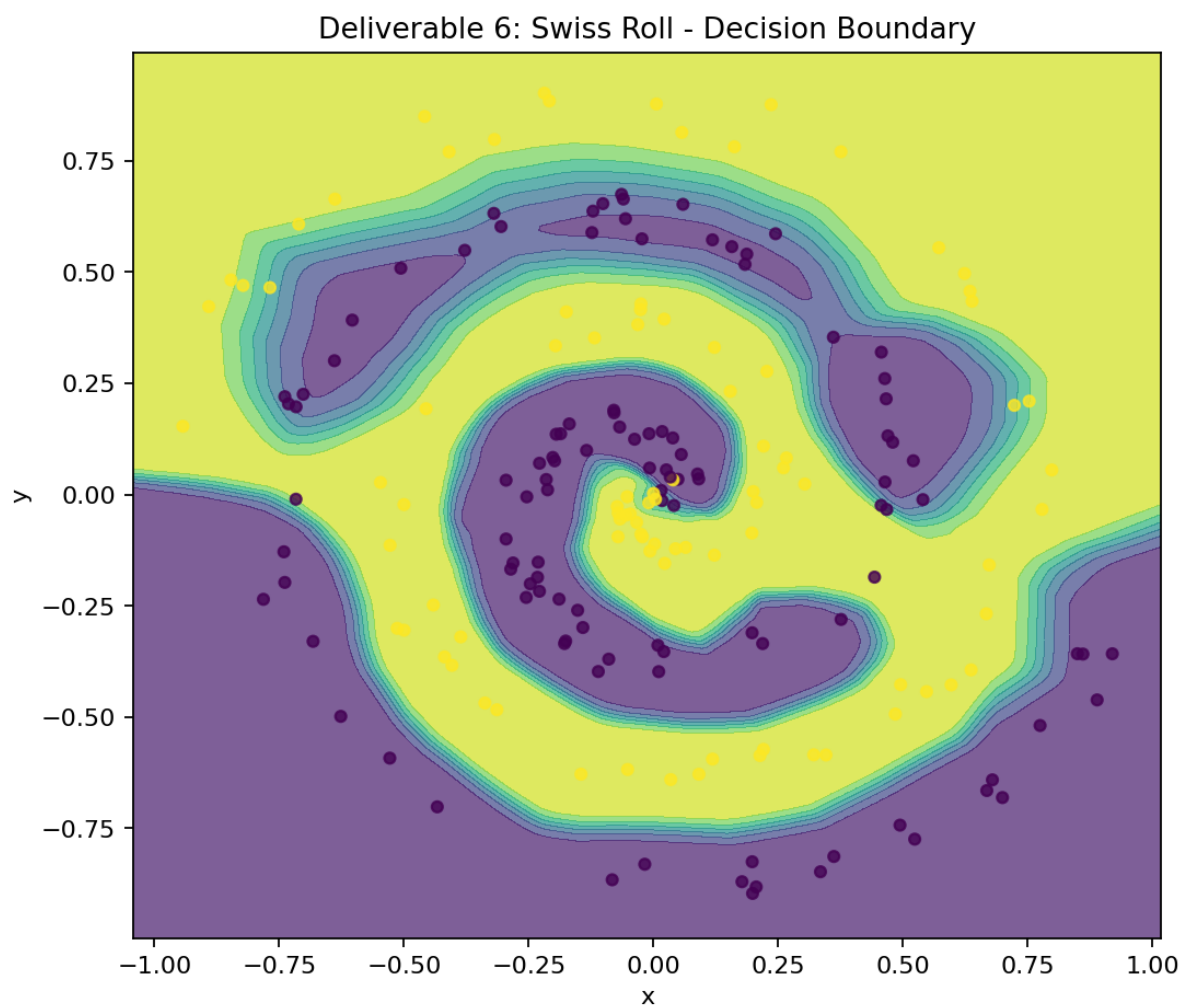


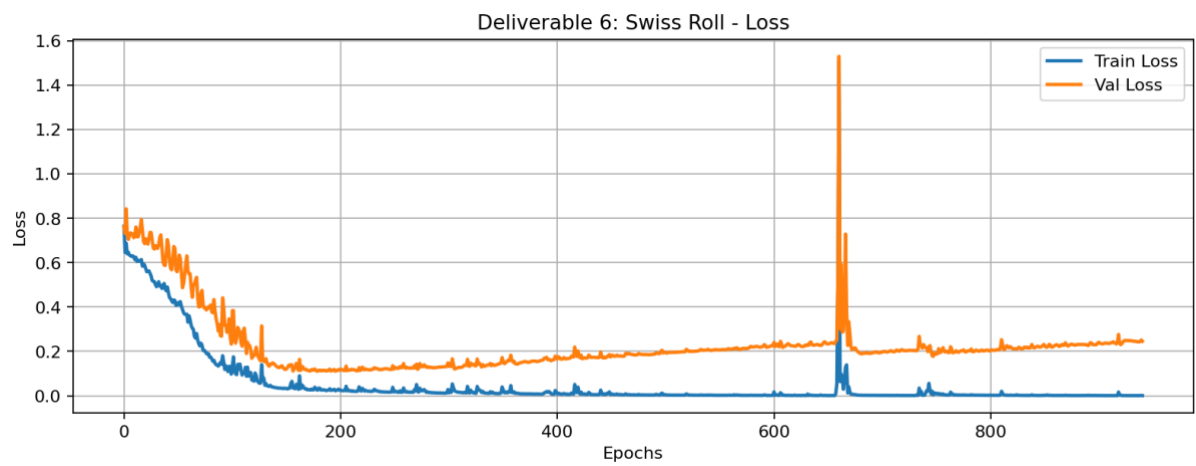
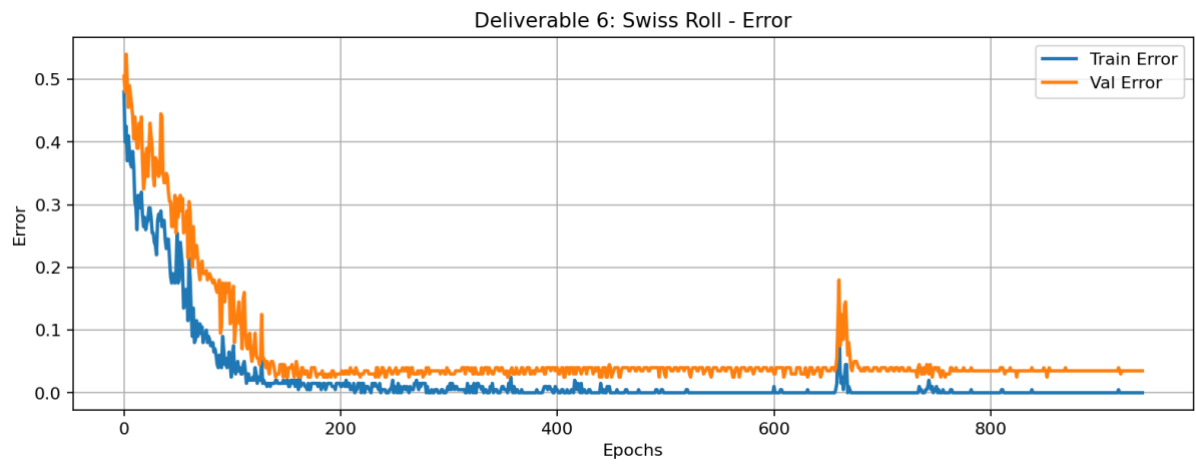
Deliverable 5C: Sinusoid (Adam) - Error





Deliverbale 6:





Deliverable 7:

